- Policies have a name and are configured in Program.cs as part of the call to `AddAuthorization()` in `ConfigureServices`. You define the policy using `AddPolicy()`, passing in a name and a lambda that defines the claims needed.
- You can apply a policy to an action or Razor Page by specifying the policy in the authorize attribute; for example, `[Authorize("CanAccessLounge")]`. This policy will be used by the `AuthorizationMiddleware` to determine whether the user is allowed to execute the selected endpoint.
- In a Razor Pages app, if an unauthenticated user attempts to execute a protected action, they'll be redirected to the login page for your app. If they're already authenticated but don't have the required claims, they'll be shown an "access denied" page instead.
- For complex authorization policies, you can build a custom policy. A custom policy consists of one or more requirements, and a requirement can have one or more handlers. You can combine requirements and handlers to create policies of arbitrary complexity.
- For a policy to be authorized, every requirement must be satisfied. For a requirement to be satisfied, one or more of the associated handlers must indicate success, and none must indicate explicit failure.